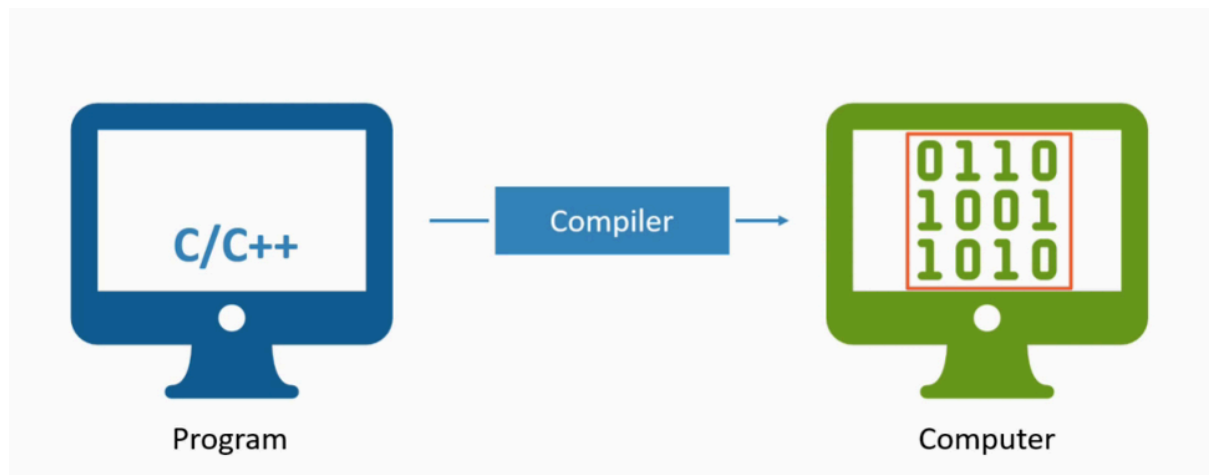


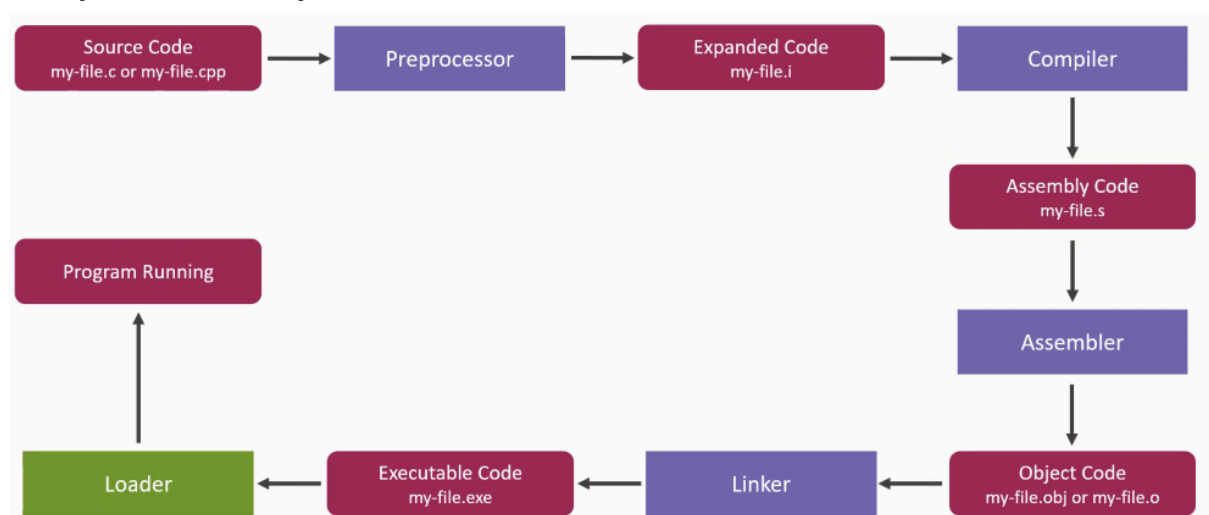
Introduction to Programming

Compilation Process



It is process where source code is converted into machine code and the output is generated of the source code

Steps of Compilation



Step 1:- Source Code (That the programmer write)

Step 2 :- Preprocessor

→ Removes Comment

→ Expand the preprocessor directive(macros/file inclusion)

For Example:-

This is the code -->

```
/*  
    Objective: How Does a C/C++ Program Run Behind a Scene?  
*/  
  
#include <iostream>  
  
#define PI 3.14  
  
double area(float);  
  
int main() {  
    double a = area(10);           // Function call to calculate area  
    return 0;  
}  
  
double area(float radius) {  
    return PI * radius * radius;  
}
```

Comment will be removed First like this -->

```
#include <iostream>  
  
#define PI 3.14  
  
double area(float);  
  
int main() {  
    double a = area(10);  
    return 0;  
}  
  
double area(float radius) {  
    return PI * radius * radius;  
}
```

Then lastly define macro will be define pi and
iostream file will be included -->

```
.....  
..... <iostream> file code .....  
.....  
  
double area(float);  
  
int main() {  
    double a = area(10);  
    return 0;  
}  
  
double area(float radius) {  
    return 3.14 * radius * radius;  
}
```

Step 3:- Compiler

- Check Syntax Error
- Convert into Assembly Code

Step 4:- Assembler

- Convert assembly code into Object Code(Binary Code/Machine)

Step 5:- Linker

- Convert Object Code into Executable File
- Generate Executable File

Step 6 :- Loader

- Load the executable file in memory
- And our program start running

Binary Language

Binary code is the fundamental language of computing, representing data using only two digits: 0 and 1 (bits). It translates text, instructions, and images into high/low electrical signals (on/off) within a computer's processor, organizing them into 8-bit groups called bytes. It is a base-2 positional numbering system.

Key Concepts of Binary Code

- **Bits and Bytes:** A single "0" or "1" is a bit. Eight bits together form a byte, which is commonly used to represent a single character, such as a letter or number.
- **Base-2 System:** Unlike our daily decimal system (base-10), binary uses powers of two. Positions (from right to left) represent 2^0 (1), 2^1 (2), 2^2 (4), 2^3 (8), and so on.
- **Hardware Application:** A '1' indicates a transistor is ON (electricity passing), while '0' indicates OFF (no electricity).
- **How it Works:** To represent the number 13 in binary, it is written as 1101 ($1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$).

→**Text Encoding (ASCII):** Text is converted into binary using standards like ASCII, where each letter corresponds to a specific 8-bit sequence (e.g., 'A' is 01000001).

IDE

An integrated development environment (IDE) is a software application that helps programmers develop software code efficiently. It increases developer productivity by combining capabilities such as software editing, building, testing, and packaging in an easy-to-use application. Just as writers use text editors and accountants use spreadsheets, software developers use IDEs to make their job easier.

Understanding First C++ Program

Code -->

```
#include <iostream>
using namespace std;
int main(){
    cout<<"Hello World :)" << endl;
    return 0;
}
```

Understanding

- #include <iostream>**: This is a preprocessor directive. It tells the compiler to include the iostream header file, which contains the necessary tools for input and output operations (like cout).
- using namespace std**:: This line tells the compiler to use the standard (std) namespace. Without this, you would have to write std::cout instead of just cout. It acts like a "shortcut" for common C++ commands.
- int main()**: This is the entry point of every C++ program. When you run the code, the computer starts executing instructions from the first line inside this function.
- { ... }**: The curly braces mark the beginning and end of a block of code. Everything inside these braces belongs to the main function.
- cout << "Hello, World!";**:

- ◆ `cout` (pronounced "see-out") is the standard output object.
- ◆ `<<` is the insertion operator, which "pushes" the text into the output stream.
- ◆ `;` (Semicolon) acts like a period at the end of a sentence; it tells C++ that the instruction is finished.

→**endl**: This stands for "end line." It moves the cursor to the next line on the screen.

→**return 0**:: This signals that the program has finished successfully. The value 0 tells the operating system that everything ran as expected.

Variables

variable is a named storage location in memory used to hold data that a program can manipulate during execution. Every variable has a specific **data type**, which determines the size of memory it occupies and the range of values it can store.

1. Basic Declaration and Initialization

To use a variable, you must first declare it by specifying its type and name. You can also assign an initial value (initialization).

- **Declaration:** `int age;` (Tells the compiler to reserve memory for an integer).
- **Initialization:** `age = 25;` (Assigns a value to the reserved memory).
- **Combined:** `int age = 25;` (Declares and initializes in one step).

2. Naming Rules (Identifiers)

Variable names in C++ must follow these rules:

- Can contain letters, digits, and underscores (_).
- **Must start** with a letter or an underscore, not a digit.
- **Case-sensitive:** `myVar` and `myvar` are different.
- Cannot use **reserved keywords** (e.g., `int`, `return`, `class`).

Data Types

Data types specify the type of data that a variable can store. Whenever a variable is defined in C++, the compiler allocates memory for that variable based on the data type with which it is declared. Please note that every data type may require a different amount of memory.

→Character Data Type (char)

The **character data type** is used to store a single character. The keyword used to define a character is `char`. Its typical size is 1 byte, and it stores characters enclosed in single quotes (' '). It can generally store upto 256 characters according to their ASCII codes.

→Integer Data Type (int)

Integer data type denotes that the given variable can store the integer numbers. The keyword used to define integers is **int**. Its typical size is **4-bytes** (for 64-bit) systems and can store numbers for binary, octal, decimal and hexadecimal base systems in the range from **-2,147,483,648** to **2,147,483,647**.

→ Boolean Data Type (bool)

The [boolean data type](#) is used to store logical values: true(1) or false(0). The keyword used to define a boolean variable is **bool**. Its typical size is 1 byte.

→ Floating Point Data Type (float)

Floating-point data type is used to store numbers with decimal points. The keyword used to define floating-point numbers is **float**. Its typical size is 4 bytes (on 64-bit systems) and can store values in the range from **1.2e-38** to **3.4e+38**.

→ Double Data Type (double)

The **double data type** is used to store decimal numbers with higher precision. The keyword used to define double-precision floating-point numbers is **double**. Its typical size is 8 bytes (on 64-bit systems) and can store the values in the range from **1.7e-308** to **1.7e+308**

→ Unsigned

Restricts the type to only positive values, effectively doubling the positive range.

How Data is Stored - Short Notes

1. Data Storage in Memory

- All data is stored in binary (0s and 1s).
- Data type decides how binary is interpreted (int, char, etc.).

2. Positive Numbers

- Stored in normal binary form.
- Example: 5 = 00000101
- Printed directly as stored.

3. Negative Numbers (2's Complement)

Steps:

Step 1:- Convert to binary

Step 2:- Take 1's complement (flip bits)

Step 3:- Add 1

- Example (-5):

5 = 00000101

Flip = 11111010

+1 = 11111011

4. Characters (ASCII)

- Characters stored as ASCII values.
- Example: 'A' = 65 = 01000001
- Printing char → character
- Casting to int → ASCII value

Operators

→Arithmetic Operator

Used for mathematical calculations

Operators :- +, -, *, /, %

→Relational Operators

Used to compare values

◆ **== :- Equal to**

◆ **!= :- Not Equal to**

◆ **> :- Greater than**

◆ **< :- Less than**

◆ **>= :- Greater than equal to**

◆ **<= :- Less than equal to**

→Logical Operators

Used to combine conditions

◆ **&& :- And ==** If both are true than true
otherwise false

◆ **|| :- OR ==** If one is true than true otherwise
false

◆ **! :- Not ==** reverse result